

Analysis of Algorithms

Assignment - 1 / Revised Unique Handwritten Copy Guide

Student Name: M. Talha

SAP Id : 64932

Course: Analysis of Algorithms

Q1: Find the largest & smallest elements?

1: Algorithm strategy:-

The strategy is to examine the array from left to right. At the start, the first element is considered both the largest & smallest. Each next element is compared with the current largest & current smallest. The corresponding value is updated whenever a larger or smaller element is found. Thus, one complete pass through the array is enough.

2: Pseudocode:-

Algorithm Find values ($A, N, \text{Max}, \text{Min}$).

(This algorithm finds the maximum & minimum value in an array).

Set:

Step - 1 (initial values)

$\text{Max} = A[0]$

$\text{Min} = A[0]$

Step 2 [Scan the array]

For $I = 1$ to $N-1$

If $(A[I] > \text{Max})$ Then

$\text{Max} = A[I]$

End If

If $(A[I] < \text{Min})$ Then

$\text{Min} = A[I]$

End If

Step - 3 [Show answers]

Write (Max, Min)

Step - 4 [End]

Exit

3- Major Primitive Operations:-

The imp primitive operations are array access $A[I]$, comparisons with Max & Min ,

assignments when a new value is found & loop/increment operations.

4:- Running Time:-

The loop checks the remaining $n-1$ elements. The same scan is required regardless of the arrangement of the values, because the algorithms must examine every element to guarantee the max & minimum.

Best case: $\Theta(n)$

Worst case: $\Theta(n)$

5:- Asymptotic Notation:-

Big- $O = O(n)$

Big- $\Omega = \Omega(n)$

Big- $\Theta = \Theta(n)$

6:- Space Complexity:-

Only Max, Min & the loop variable i are extra variables. Therefore the auxiliary space is $O(1)$.

Q2:- Search for a Given value X :-

7:- Two different Strategies:-

Strategy 1:- Linear Search:-

Start from the first element & compare each element with X until X is found or the array ends. The array does not need to be sorted.

Strategy 2 :- Binary Search:-

Use the middle element to decide which half can contain X . This strategy can only be used when the array is sorted.

8:- Linear Search Pseudocode:-

Algorithm check value
(A, N, X).

{ This algorithm checks whether X is present in an Array }

2)

Name: M. Talha

Sap Id: 64932

```
Step 1 { Check each position }.  
For  $I = 0$  to  $N-1$   
If  $(A[I] = X)$  then  
Write ("Value Found").  
Exit  
End If
```

```
Step 2 { Value is absent }  
Write { Value not found }.
```

```
Step -3 [End]  
Exit.
```

9:- Linear Search Complexity:-

Best case:- X is at the first position, so only one comparison is made $\Theta(1)$

Worst case:- X is at the last position or absent, so n comparisons may be made $\Theta(n)$

Overall upper bound:- $O(n)$ The minimum possible running time is $\Omega(1)$

Space Complexity :- $O(1)$

10:- Binary Search Pseudocode:-

Algorithm Half search ($A, N, X,$
 $left, right, Middle$):

{ This algorithm searches X in
a sorted array by reducing
the search range }

Step 1 { Set Search range }
 $left = 0$
 $Right = N - 1$

Step 2 { Repeat search }
While ($left \leq Right$).
 $Middle = (left + Right) / 2$
If ($A[Middle] = X$) Then
Write (Value found)
Exit

Else If ($X < A[Middle]$) Then
 $Right = Middle - 1$

Else
 $left = Middle + 1$
End if
End while

Step -3 { Value is absent }
Write { "Value Not found" }

Step -4 { End }

Exit

Q3:- Complexity & Comparison

II. Binary Search Complexity:-

Best case:- X is the Middle element during the first check, so $O(1)$

Worst case:- The search interval becomes half after every comparison, giving $O(\log n)$

Overall upper bound:- $O(\log n)$ The minimum possible running time is $\Omega(1)$

Space complexity:- $O(1)$ b/c the algorithm is iterative.

12:- Comparison of Both Algorithms:-

Point	Linear Search	Binary Search
• Condition	Sorted or unsorted	Sorted array required.
• Best case	$O(1)$	$O(1)$
• Worst case	$O(n)$	$O(\log n)$
• Main idea	check one by one	Divide search range in half

Useful when	Small / unsorted data or one search	large sorted data or repeated searches.
-------------	-------------------------------------	---

13 :- Effect of Input Size & Arrangement :-

In linear Search, the arrangement of elements directly affects the number of comparisons. If X is near the beginning, the algorithm can finish quickly. If X is near the end or does not exist, most of all elements are checked. Therefore its worst-case time grows linearly as n becomes larger.

In Binary Search, the search range is cut into half at every step. Therefore increasing n causes a much slower increase in the number of comparisons. However, this advantage is available only when the array is sorted.

Name:- M. Talha

Sap id:- 64932.

14:- Final Conclusion:-

Linear Search is simple & does not need a sorted array.

Binary Search is more efficient for large sorted data because its worst-case time is logarithmic. The choice of algorithm depends on the size & arrangement of the input & whether the data is already sorted.